

Informe: la válvula MCP informada

Fecha	25 de julio de 2026
Repositorio	<code>oxidegate-lens</code> — <code>main</code> en <code>86c8656</code>
Versiones	<code>oxidegate-lens 0.4.0</code> · <code>OxideGate 0.3.1</code> · <code>OpenCode 1.18.5</code>
Estado	238 tests en verde · 28 commits · 2 binarios publicados
Seguimiento	Project «OxideGate + Lens»

Todas las cifras de este documento están **medidas**, no estimadas. Cada una procede de una ejecución real contra un OxideGate corriendo, del snapshot que escribe `mcp-savings` en disco, o de la suite de tests. Donde algo no se pudo medir, se dice.

1. Resumen ejecutivo

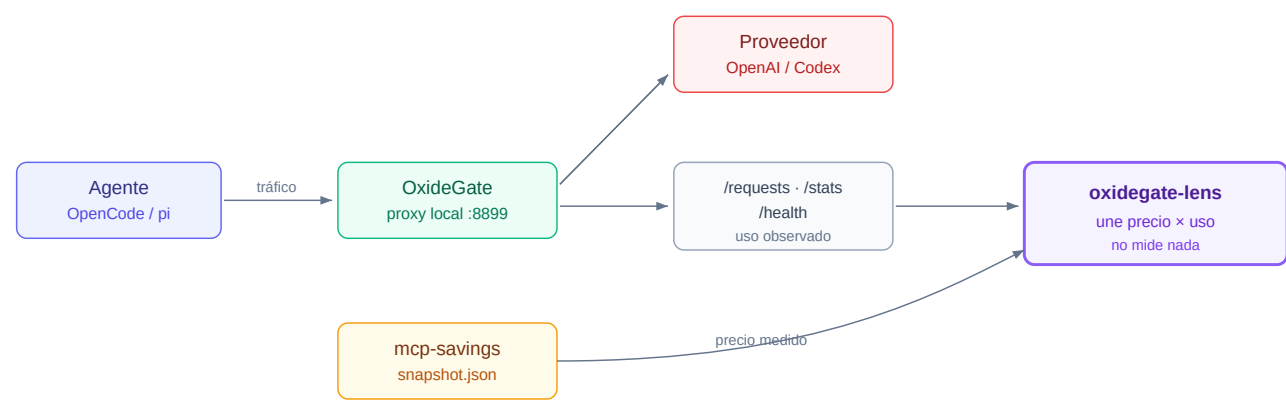
Se construyó una **válvula MCP informada**: una herramienta que dice cuánto cuesta cada servidor MCP en el cable y si desconectarlo es defendible.

Tres resultados, por orden de importancia:

1. **Funciona la mitad que mide el precio.** Sabemos, por servidor y con exactitud, cuántos bytes de esquema viajan.
2. **No puede funcionar la mitad que mide el uso** en los dialectos que usas — y eso se descubrió midiendo, no razonando.
3. **El monitor llevaba vacío desde siempre por un paquete desactualizado**, no por un fallo de código.

El punto 2 es el hallazgo del trabajo y acota el producto. El punto 3 es la razón por la que nada de esto se había podido comprobar antes.

2. Arquitectura



Dos instrumentos independientes. El precio viene de `mcp-savings`, que mide los esquemas. El uso viene de OxideGate, que observa el cable. La lens no mide nada: **une** ambos y se niega a concluir cuando no puede.

Esa separación es lo que hace posible la honestidad del sistema: cuando los dos instrumentos no se corresponden, hay una tercera respuesta posible además de "sí" y "no", que es **"no lo sé"**.

Módulos

Módulo	Responsabilidad	Puro
<code>lib/mcp-snapshot.mjs</code>	Lee el precio del snapshot	✓
<code>lib/mcp-usage.mjs</code>	Observa el uso en el cable	✓
<code>lib/mcp-valve.mjs</code>	Une precio × uso → recomendación	✓
<code>lib/mcp-protection.mjs</code>	Qué está protegido y qué se puede tocar	✓
<code>lib/mcp-transitions.mjs</code>	Diff entre dos lecturas de estado	✓
<code>lib/mcp-notices.mjs</code>	Los mensajes que lee el usuario	✓
<code>lib/mcp-config-editor.mjs</code>	El selector: estado, teclas y render	✓
<code>lib/mcp-project-config.mjs</code>	Config por proyecto y su modelo de confianza	✓
<code>lib/mcp-doctor.mjs</code>	Diagnóstico de la cadena completa	✓
<code>bin/oxidegate-savings.mjs</code>	Reporte por terminal	—
<code>bin/oxidegate-mcp.mjs</code>	Selector interactivo	—
<code>opencode/oxidegate-lens.ts</code>	Adaptador fino sobre <code>lib/</code>	—

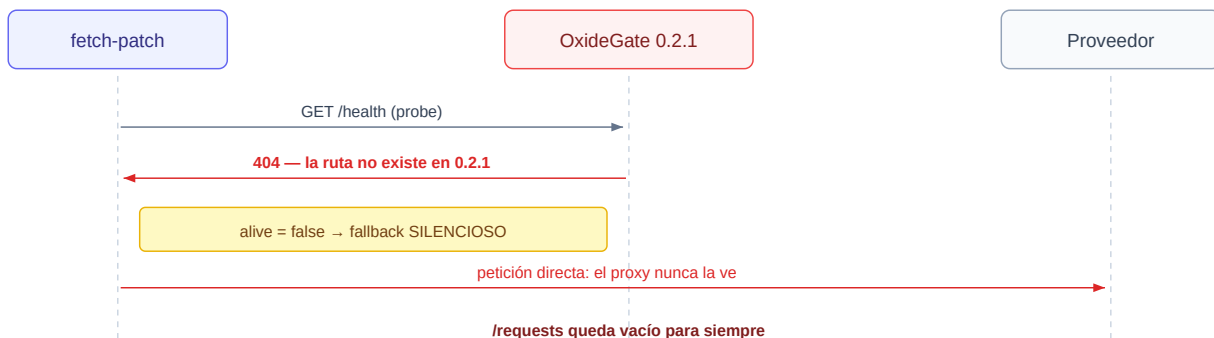
Decisión de diseño 1: el plugin es TypeScript y no se puede ejecutar con el runner del repo (`node --test` sobre `.mjs`). Por eso **toda la lógica vive en `lib/`**, que sí es testeable, y el plugin solo cablea. Los tests estáticos fallan si esa lógica empieza a filtrarse de vuelta al plugin.

La misma regla se aplicó después a la TUI del selector, que es todavía menos testeable que el plugin: modo raw de teclado, secuencias de escape, un terminal que solo existe cuando hay alguien mirando. `renderEditor` devuelve un string en vez de pintar, y `bin/oxidegate-mcp.mjs` se queda con las tres cosas que ninguna función pura puede hacer — leer teclas, escribir en pantalla y tocar el disco.

Y la regla se rompió tres veces en ese mismo fichero, lo que dice más de su necesidad que cualquier argumento: primero la fusión del `config.json`, después el pie con la ruta, y en ambos casos la lógica nació dentro del binario y hubo que moverla. El guardarraíl estático no es ceremonia; es lo único que se ha dado cuenta.

3. La cadena que estaba rota

El monitor no mostraba nada. La causa resultó no ser código, sino un paquete desactualizado, y estuvo escondida detrás de un fallo silencioso por diseño.



`/health` existe desde **OxideGate 0.3.0** — verificado por conteo: `git show v0.2.1:src/main.rs | grep -c /health → 0`; contra `v0.3.0 → 2`. El código nunca faltó. Faltaba publicar el release: el tap de Homebrew servía 0.2.1 mientras el repo iba por 0.3.0.

Por qué costó tanto verlo: el fallback es silencioso *a propósito* — el comentario del patch dice *"the pre-flight probe is what makes the fallback safe"*. Degrada a tráfico directo sin error y sin log. Y nada en la cadena reporta un desajuste de versión.

Lección general: un *pin* de paquete obsoleto es invisible de una forma en la que una función ausente no lo es. El código fuente tiene el arreglo, los tests pasan, el tag existe — y el usuario sigue ejecutando algo de hace meses. Comprueba qué está **instalado** antes de concluir que algo no está implementado.

El segundo fallo: token correcto, puerta equivocada

Una vez arreglado el probe, apareció otro error que parecía de permisos:

```
Missing scopes: api.responses.write
```

No era un problema de permisos. OxideGate expone **dos** rutas Responses que reenvían a sitios distintos, y el fetch-patch apuntaba a la equivocada:

Ruta	Reenvía a	Credencial que espera
/v1/responses	api.openai.com	API key con scopes
/v1/codex/responses	chatgpt.com/backend-api/codex	OAuth de suscripción

El patch capturaba tráfico de Codex (OAuth) y lo entregaba a la API de pago por token. Evidencia medida, antes y después de cambiar una línea:

```
ANTES /v1/responses      → openai    14 peticiones    500×10    503×2    401×2
DESPUÉS /v1/codex/responses → codex      6 peticiones    200×6
```

Lección general: un error de autenticación puede ser un error de enrutado. Comprueba a qué upstream reenvió el proxy antes de creerte un mensaje de permisos.

4. El hallazgo principal: `tools_flattened`

Con el tráfico entrando por fin, la primera ejecución real contra el cable reveló el límite del producto.

En la ruta `/v1/responses` (y también en `/v1/codex/responses`, mismo dialecto), **OpenCode aplan**a todas las tools en un único bloque sin atribución:

```
{
  "tools_by_server": [{ "server": "(native)", "kind": "native", "tools": 40 }],
  "tools_flattened": true,
  "context_tools_bytes": 48172
}
```

Las tools MCP **están en el cable** — son parte de esas 40 — pero nada dice de qué servidor viene cada una.

Techo levantado — ver §10. Esa última frase describía el estado de entonces. OxideGate publica ahora los **nombres** de las tools observadas (`tools_by_server[].tool_names`) y la lens los cruza contra la lista autoritativa del snapshot. El cable sigue sin decir **de quién** es cada tool; dice **cuáles** viajaron, y eso basta para que lo resuelva quien tiene el inventario.

La prueba de que las MCP están dentro

Al conectar y desconectar servidores MCP, el conteo y el peso del bloque `(native)` se mueven juntos, sin que aparezca jamás un segundo servidor:

tools	bytes	bloque reportado
34	39985	(native)
35	42116	(native)
40	48172	(native)

Tres estados del mismo sistema, medidos en la misma sesión. El cable las transporta; el dialecto no las distingue.

Consecuencia para el producto

Mitad de la válvula	Estado
Precio — cuánto cuesta cada servidor	✓ Funciona
Uso — cuántas veces se usó cada servidor	✗ Imposible en rutas que aplanan

Esto **no es un bug**. Es el dominio de aplicación real, y era imposible conocerlo sin llegar a medir.

El error que casi se envía

La lens no conocía el campo, así que caía en `insufficient-observation`, cuya acción implícita es **esperar**. En una ruta que aplanan, esperar no sirve nunca. El usuario habría acumulado tráfico indefinidamente esperando una respuesta que no puede llegar.

Corregido: `tools-flattened` **gana** a `insufficient-observation`, porque uno es permanente y el otro transitorio.

Y hubo un segundo defecto, más sutil, que solo se vio leyendo la salida real: bloquear la recomendación **no bastaba**, porque la fila seguía imprimiendo `0 usos`. Con las tools aplanadas ese cero deja de significar "no se usó" y pasa a significar "no se pudo saber".

Lección general: la invariante de honestidad puede sostenerse en la **decisión** y romperse en el **render**. Hay que comprobar las dos capas.

5. Los precios medidos

Datos del snapshot de `mcp-savings` del 2026-07-25T09:53:14Z (fresco):

Servidor	Bytes de esquema	Tokens	Peso relativo
engram	17 233	3 788	79 %
context7	4 577	977	21 %
Total MCP	21 810	4 765	

Puestos en contexto contra lo que realmente viaja en cada petición:

Esquemas de tools por petición (bytes)

OpenCode /v1/responses	48 172	
pi / Codex	72 570	
— de los cuales MCP —	21 810	(medible, pero no atribuible en el cable)

Dato que conviene mirar dos veces: `pi` gasta **72,5 kB de esquemas nativos por petición** — más del triple que todo el MCP junto. En `pi`, la pregunta interesante no es "qué MCP desconecto" sino "por qué viajan 72 kB de tools nativas en cada llamada".

Medición de `pi`

7 peticiones reales capturadas en la telemetría. Todas idénticas en forma:

```
tools_by_server = ["(native):native"]    ← en las 7, sin excepción
context_tools_bytes = 72570
```

Cero servidores MCP en el cable, igual que OpenCode.

6. Las invariantes de honestidad

Este es el eje del diseño, y lo que hace que la herramienta sea confiable.

6.1 Una ausencia nunca es un cero

Rige todos los módulos. Un servidor que no se pudo medir se renderiza como `desconocido`, jamás como `0`. La razón es concreta: **un cero fabricado es lo que hace que una herramienta recomiende desconectar algo que estás usando**.

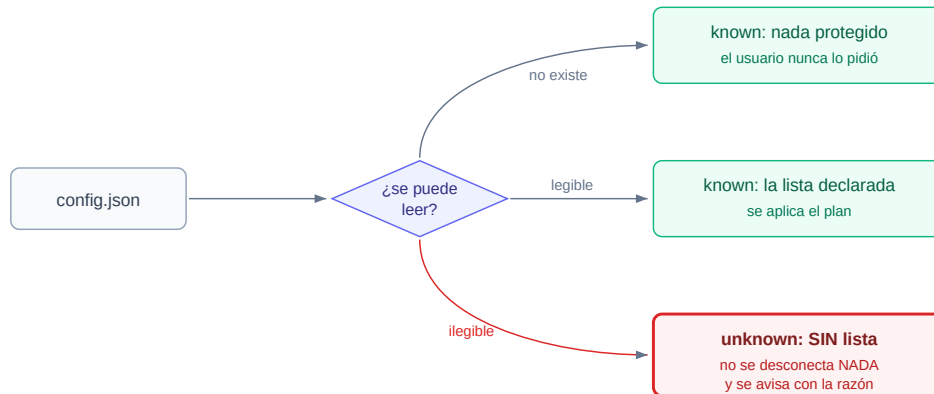
El caso que lo demuestra: `mcp-savings` construye `bytes` como suma de una lista de tools. Cuando un servidor falla al conectar, esa lista está vacía, y `bytes` sale **presente, numérico y 0** — un campo real con un número sin significado. Solo el flag `ok` distingue un cero real (`ok: true, bytes: 0`, un servidor sin tools) de uno inmedible (`ok: false, bytes: 0`).

6.2 La misma regla, apuntando al revés

En la configuración de protección la invariante se invierte, porque ahí el "cero" no es una cifra en un informe: **es una orden**.

Una lista de protegidos vacía significa "desconecta todo".

Un lector que degradara un JSON roto a `[]` desconectaría exactamente los servidores que ese fichero existía para proteger. Por eso un fichero ilegible devuelve `status: 'unknown'` y **no lleva array alguno** — no puede haber nada que un llamador descuidado pueda iterar.



6.3 Una ventana de observación siempre acompaña a su cifra

Ningún `0` usos ni `candidato a desconectar` se imprime sin la ventana observada **en la misma línea**. Hay un assert a nivel de render que lo prohíbe.

6.4 Una lista parcial es peor que una admitida

Si la lista de protegidos trae `["engram", 42, "context7"]`, se podría filtrar el `42`. **No se hace**. Una lista filtrada parece completa al llamador, que no tiene forma de distinguirla de una intacta — y la entrada descartada es un servidor que luego se desconecta.

7. Qué está verificado y qué no

Distinción deliberada. Un "verificado" general sería tan falso como un "no verificado" general.

Verificado contra sistemas reales

Comprobación	Cómo	
<code>client.mcp.status</code>	Sesión OpenCode 1.18.4 real	
<code>client.mcp.disconnect</code>	Idem — deja el servidor en <code>"disabled"</code>	
<code>client.mcp.connect</code>	Idem — restaura a <code>"connected"</code>	
Aviso de arranque	Toast real: <i>"empiezas con 1 MCP sin conectar: context7"</i>	
Protección de servidores	El toast dijo 1 , no 2 — <code>engram</code> sobrevivió	
<code>session.idle</code> dispara	Conexión manual → aviso en el siguiente reposo	
Silencio sin cambios	Los reposos vacíos no produjeron ningún aviso	
Pipeline completo	Ejecutado contra snapshot real y <code>/requests</code> real	
Aviso de conexión	Conexión manual → aviso en el siguiente reposo. <code>session.idle</code> sí dispara	
Silencio sin cambios	Los reposos sin movimiento no produjeron ningún aviso	
Interruptor en config	Plugin conducido sin ninguna variable de entorno	
Selector sin TTY	Degrada a imprimir el estado; <code>--help</code> responde; <code>`</code>	head` no revienta
Interruptor en config	Plugin conducido sin ninguna variable de entorno	

Detalle que solo aparece ejecutando: tras un `disconnect` , el SDK devuelve el estado `"disabled"` , no `"disconnected"` . El plugin sobrevive a eso únicamente porque pasa el estado `verbatim` y nunca lo compara contra una cadena fija.

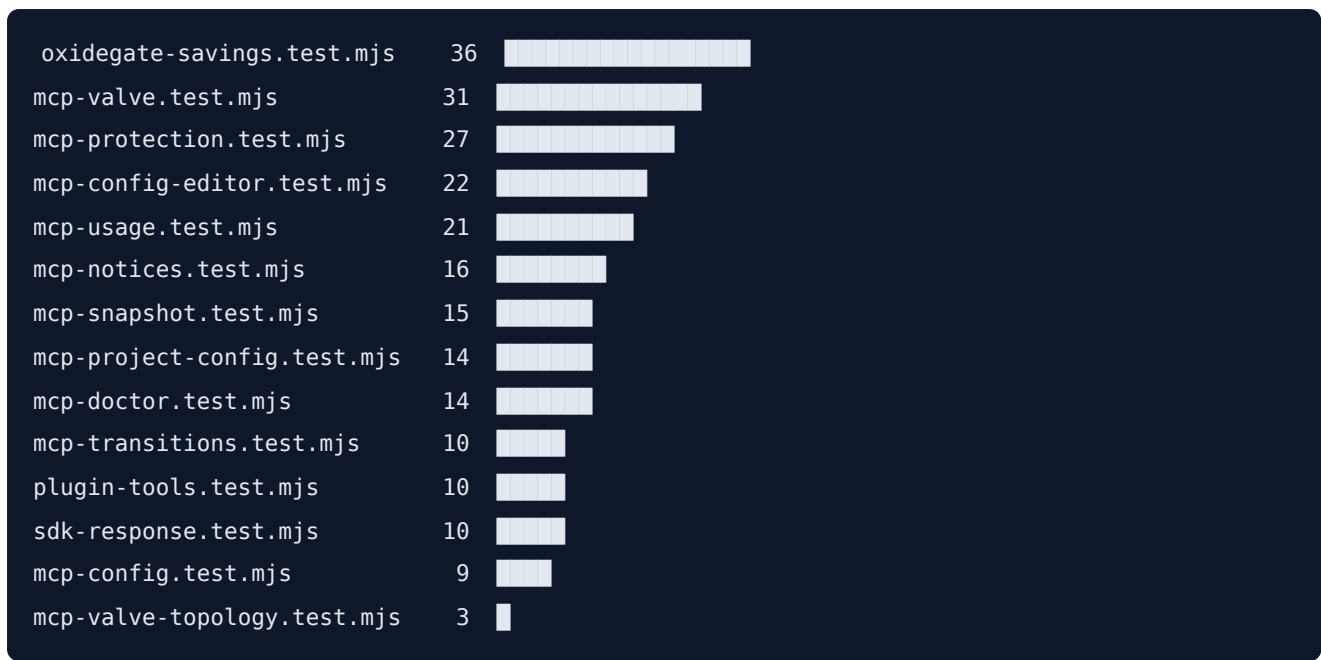
No verificado

Qué	Por qué importa
Hook <code>tool.execute.after</code>	Solo ejercitado con cliente falso: prueba que el cableado corre, no que OpenCode lo dispare
Mitad uso de la válvula	Nunca ha podido ejercitarse: todas las rutas medidas aplanan

8. Estado del código

Tests

238 tests, 0 fallos. Distribución:



Prueba de mutación

Norma de la casa: **un test que no puede fallar es decoración**. Cada invariante crítica se sometió a una mutación que debía matarla:

Mutación aplicada	Test que murió
Quitar la guarda de colisión del recomendador	CLI <code>name-collision</code>
Colapsar la colisión a su primera entrada	CLI <code>name-collision</code>
Degradar config rota a lista vacía	<code>malformed_json_is_unknown</code>
Filtrar entradas no-string en vez de rechazar	<code>non-string entries</code>
<code>unknown</code> de protección → "nada protegido"	<code>planMcpDisable refuses</code>
Primera lectura como comparación contra vacío	<code>first reading is baseline</code>
Servidor que aparece contado como conexión	<code>APPEARED is not connected</code>
Emitir aviso sin que nada se moviera	regla del silencio
Permitir el <code>0</code> con tools aplanadas	<code>uses es undefined</code>
Quitar la precedencia del aplanado	<code>flattened outranks</code>
Coercer el interruptor en vez de exigir boolean	<code>non-boolean is UNKNOWN</code>
Override que solo puede encender	<code>env can also turn it OFF</code>
Snapshot ausente degradado a lista vacía	<code>absent snapshot is NOT an empty list</code>
Descartar protegidos que el snapshot no conoce	<code>protected name ... still gets a row</code>
Sobrescribir el config en vez de fusionar	<code>conserva claves ajenas</code>
Aplicar un config de proyecto sin aprobarlo	3 tests del modelo de confianza
Aprobar por RUTA en vez de por contenido	<code>si el CONTENIDO cambia, vuelve a pendiente</code>

Diecisiete mutaciones, cada una mató **exactamente** su test y ningún otro.

Commits

28 commits, cada uno una unidad de trabajo verificada en verde por separado.

9. Configuración

Todo en un solo fichero, `~/config/oxidegate-lens/config.json`:

```
{
  "disableByDefault": true,
  "protectedMcpServers": ["engram"]
}
```

Al abrir OpenCode:

```
empiezas con 1 MCP sin conectar: context7 (4.6 kB). Siguen activos: engram (17.2 kB).  
Para volver a abrir alguno: oxidegate_lens_mcp_connect. Detalle completo:  
oxidegate_lens_mcp_valve.
```

Ver el estado, **medir** el coste y **saber cómo revertirlo**, sin ejecutar ningún comando.

El selector: elegir sin editar JSON

```
oxidegate-mcp
```

```
Servidores MCP – elige cuáles se preservan al arrancar
```

```
> ● engram          17.2 kB   se preserva  
  ○ context7        4.6 kB   se desconecta al arrancar
```

```
Desconectar al arrancar: ACTIVADO
```

```
Se guarda en: ~/.config/oxidegate-lens/config.json
```

```
Lo aplica: el plugin de OpenCode. En pi, Codex CLI u otros harnesses  
este fichero se guarda pero HOY no lo lee nadie.
```

```
↑↓ mover · espacio preservar/desconectar · d interruptor · enter guardar · q salir
```

Con **el precio al lado de cada servidor**, que es la mitad del valor: no se elige a ciegas, se elige sabiendo qué cuesta cada uno.

Funciona **sin OpenCode**, a propósito. La configuración es del usuario, no del harness, así que sirve igual con `pi` o con lo que venga. Lo que **no** puede hacer es tocar una sesión en marcha: eso necesita el SDK del harness y vive en el plugin.

Tres reglas que el módulo defiende, y las tres son la misma invariante de siempre aplicada a una pantalla:

- **Un snapshot ausente no es una lista vacía.** Pintar un selector vacío diría «*no tienes MCPs*», que es una afirmación. Lo único que se sabe es que no se pudo leer el fichero, así que se dice eso y se sale con código 1.
- **Un nombre protegido que el snapshot no conoce sigue saliendo**, marcado como sin medir y sin precio inventado. Servidor apagado y nombre mal escrito son indistinguibles desde ahí — pero borrarlo de la pantalla lo borraría del fichero al guardar.
- **Marcar protegidos con el interruptor apagado no protege de nada**, así que la pantalla lo dice en vez de dejar rellenar una lista que hoy no hace nada.

Y se guarda **fusionando**: si el fichero tiene otras claves, no son nuestras y no se tocan.

El fallo silencioso que tuvo, y por qué era el peor posible

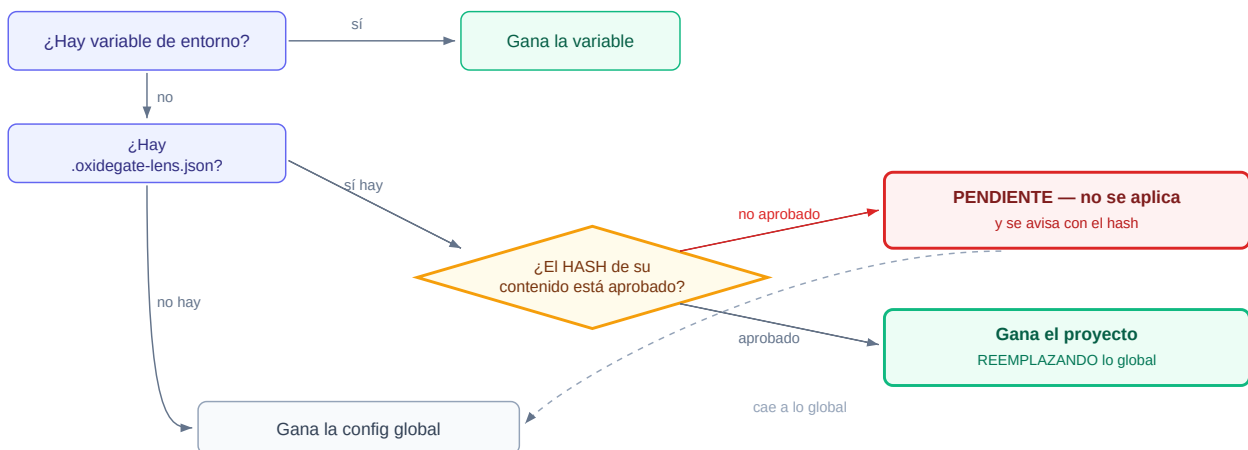
La primera versión decía **dónde** se guarda la configuración y nunca **quién la lee**. En `pi` o en Codex CLI eso significaba marcar servidores, ver la ruta, y creer que se había configurado algo — cuando el único consumidor de ese fichero es el plugin de OpenCode.

Guardaba bien y no aplicaba nada. **Sin error que buscar**, que es la peor forma de fallar: no hay nada que investigar, solo una impresión falsa. Las dos líneas del pie de arriba son el arreglo, y van dentro de `renderEditor` para que sea imposible pintar el selector sin ellas.

Configuración por proyecto, y por qué no se aplica sola

Un proyecto puede declarar la suya en `.oxidegate-lens.json`. **Reemplaza a la global en lo que declara**, no se fusiona — se lee un fichero y se sabe la respuesta, en vez de componer dos mentalmente. Precedencia completa:

```
variables de entorno > proyecto (si está aprobado) > global > por defecto
```



Y **no se aplica hasta aprobarlo**. Un fichero de configuración dentro de un repositorio clonado es **código ajeno**: puede desconectar servidores MCP que querías conservar, o —peor, porque es silencioso— marcar como protegido uno que querías apagar.

```
oxidegate-mcp --approve
```

Enseña el fichero **antes** de aprobarlo, porque aprobar a ciegas no es consentir. Y **la aprobación es del CONTENIDO, no de la ruta**, siguiendo el modelo de `direnv`: aprobar una ruta para siempre dejaría que el repositorio cambiara el fichero mañana —basta un `git pull`— y se aplicara sin que nadie se enterara. Verificado en vivo: aprobar lo aplica, editar el fichero lo devuelve a pendiente.

La idea de tener configuración por proyecto era nuestra y anterior. Lo que aportó estudiar `pi` fue **el problema de seguridad que esa idea arrastra, ya resuelto** — su modelo `--approve` / `--no-approve`.

La forma del resultado hace el trabajo de seguridad

Tres decisiones que no son estilo:

- Un `pending` **no trae** `config`. Un llamador que se salte la comprobación de estado no tiene nada que aplicar.
- El fichero de aprobaciones devuelve `{}` ante **cualquier** fallo: nada aprobado, nunca todo aprobado.
- **Asimetría deliberada**: en las aprobaciones se *filtran* las entradas basura; en la lista de protegidos se *rechaza* la lista entera. Filtrar una aprobación aprueba menos; filtrar una protección protege menos. La misma operación, dirección de seguridad opuesta.

Por qué el interruptor no vive en una variable de entorno

Estuvo en `OXIDE_GATE_MCP_DISABLE_BY_DEFAULT` y falló una prueba real: **el propio autor abrió OpenCode sin exportarla**, con las instrucciones delante, y la función simplemente no corrió — sin ninguna pista de que estaba apagada.

Un interruptor que hay que recordar exportar antes de arrancar un proceso está apagado la mayor parte del tiempo. Y dejaba la configuración incoherente: la lista en un fichero, el interruptor en el entorno. Dos mecanismos para una sola función.

Las dos variables siguen funcionando y **ganan** cuando están definidas, en **ambas direcciones** — un override que solo puede encender no es un override.

10. El diagnóstico: `--doctor`

Cada comprobación de este comando es un fallo que costó tiempo real encontrar y que **no era diagnosticable desde fuera**. Juntas son un comando de un segundo.

```
oxidegate-savings --doctor
```

```
✓ El proxy responde
✓ Lo que responde ES OxideGate
✓ /health responde 200
✓ 41 peticiones observadas
! Las tools llegan APLANADAS (tools_flattened)
✓ Precio disponible para 2 servidor(es)
✓ Desconectar al arrancar: ACTIVADO
✓ Sin configuración de proyecto

DEGRADED – Funciona, pero hay algo que limita lo que se puede reportar.
```

La regla que lo gobierna: una comprobación que no se pudo hacer sale como `?`, jamás como `✓`, y el veredicto no puede ser `OK` mientras quede alguna sin saber. Decir «todo bien» sobre algo que no se miró **cierra la investigación** que habría encontrado el problema.

Tres defectos que aparecieron ejecutándolo

Ninguno era visible en los catorce tests unitarios del módulo:

Defecto	Por qué importaba
El doctor moría antes de diagnosticar	<code>getJSON</code> lanza a propósito y el proceso sale: fallaba justo en los dos casos para los que existe
Se creía un 200 de un desconocido	Con un WordPress en el puerto, decía «el proxy está vivo» dos líneas después de descartar que ese servicio fuera OxideGate
El veredicto enterraba lo accionable	<code>unknown</code> pesaba más que <code>warn</code> , así que un proxy sano sin tráfico se resumía como «desconocido», escondiendo el único aviso arreglable detrás de un desconocido que era su propia consecuencia

El mock de tests sirve ahora `/health`, como el OxideGate real desde 0.3.0. Un fixture que no lo sirviera diagnosticaría un proxy sano como roto: **el fixture tiene que representar el sistema, no una versión anterior de él.**

11. La vía para levantar el techo

El límite de la §4 —que el cable no conserva la atribución— resultó **no ser insalvable**. Y llegar ahí exigió desenterrar por qué ya se había intentado.

Lo que se intentó antes, y por qué se rechazó bien

OxideGate #5 pedía exactamente esto: «*atribuye tools por servidor MCP en clientes que los aplanan*». Se cerró con el PR #9, y **la decisión fue correcta**:

OpenCode manda `enram_mem_search` separado por `_`, que colisiona con nativas como `apply_patch` o `delegation_list`. Una heurística de nombres habría misatribuido `delegation` como si fuera un servidor.

En una herramienta de honestidad eso es inaceptable. Por eso enviaron `tools_flattened`: la admisión honesta de «*no puedo atribuir*» en lugar de una atribución fabricada. Es la misma disciplina que rige todo este informe.

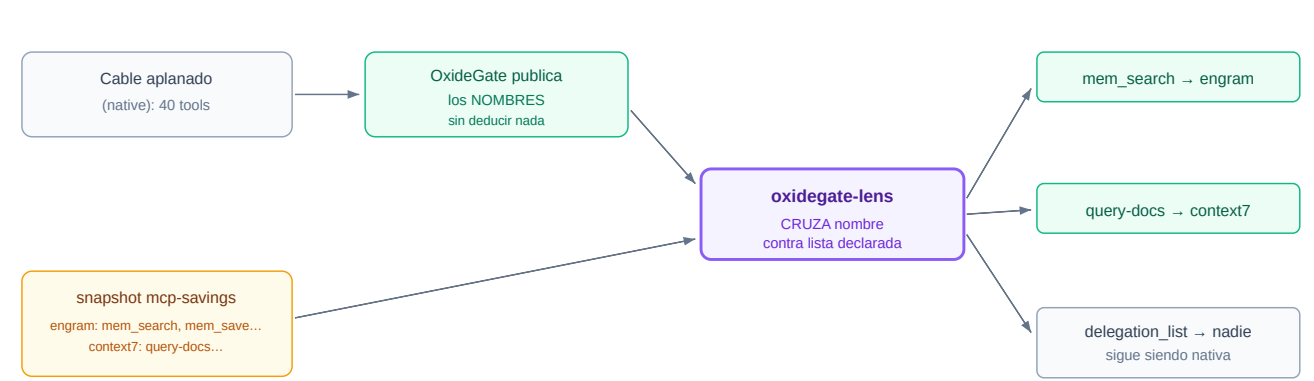
Por qué la propuesta nueva sí puede funcionar

La diferencia es entera: #5 pedía que OxideGate DEDUJERA el servidor. La propuesta actual (OxideGate#19) pide que **no deduzca nada** — que publique los nombres que ya parsea, porque es imposible contar 40 tools y sumar 48.172 bytes sin haberlos leído — y que el cruce lo haga la lens contra la lista exacta del snapshot.

Cliente	Manda en el cable	El snapshot declara	Resultado
OpenCode	enram_mem_search	enram → mem_search	casa por los dos extremos
pi	mem_search (crudo)	enram → mem_search	casa exacto
cualquiera	delegation_list	no está en ninguna lista MCP	no casa → sigue nativa

La tercera fila es justo el fallo que temían, y no puede ocurrir.

La razón de fondo por la que #5 no era resoluble tal como se planteó: le pedía a OxideGate una respuesta que no tiene datos para dar. No tiene el snapshot. Cada instrumento sabe la mitad, y la solución exige a los dos.



El campo existe en OxideGate — pero todavía no en ningún release

OxideGate aceptó la propuesta. El campo existe:

Commit	6c9b98a , PR #25, mergeado el 25/07 a las 17:17
Contrato	pub tool_names: Vec<String> en cada entrada de tools_by_server
Publicado	No. El último tag sigue siendo v0.3.1 , de las 08:53
El proxy en ejecución	tool_names: AUSENTE

Es el mismo patrón que dejó el monitor vacío durante meses: el código tiene el arreglo, el tag no lo alcanza, y quien lo usa sigue ejecutando lo anterior. Documentado en la §3 como lección, y repetido aquí como hecho.

Un detalle del contrato que la lens tendrá que respetar cuando lo consuma:

Acotada a MAX_TOOL_NAMES entradas. Si tool_names.len() < tools , la lista está truncada y el conteo tools es el bueno.

Una lista truncada **no permite concluir «este servidor no se usó»**: sus herramientas pueden haber quedado fuera del corte. Atribuir sobre ella fabricaría exactamente el cero que este proyecto existe para

no fabricar.

Implementado

`OxideGate#19` se cerró (PR#25 de OxideGate) y la lens ya lo consume. El techo de la §4 **está levantado en código**, no solo en el plan:

- `lib/mcp-snapshot.mjs` expone `toolNames` por servidor. Una lista **ausente** queda `undefined` y bloquea; `[]` se conserva como la afirmación distinta de «este servidor no declara ninguna».
- `lib/mcp-usage.mjs` **observa** los nombres del cable aplanado (`usesByToolName`) sin atribuir nada — no conoce el snapshot, y su contrato dice que no debe conocerlo.
- `lib/mcp-valve.mjs` hace el cruce (`attributeFlattenedNames`), que es una **búsqueda contra la lista declarada**, no una heurística de nombres.

Cuándo se sigue bloqueando, y por qué es estricto

El rechazo por `tools-flattened` se mantiene entero si:

Situación	Por qué bloquea
Alguna fila aplanada no trajo nombres	Atribución parcial: un cero podría venir de la petición que no supo nombrar
Ningún servidor del snapshot declara sus tools	No hay contra qué cruzar
Un nombre casa con dos servidores	Atribuirlo a uno sería inventar

Una atribución a medias es **peor que ninguna**: un cero parcial se lee como un cero medido. Misma disciplina que la fila `unknown` — lo que no se sabe se declara, no se rellena.

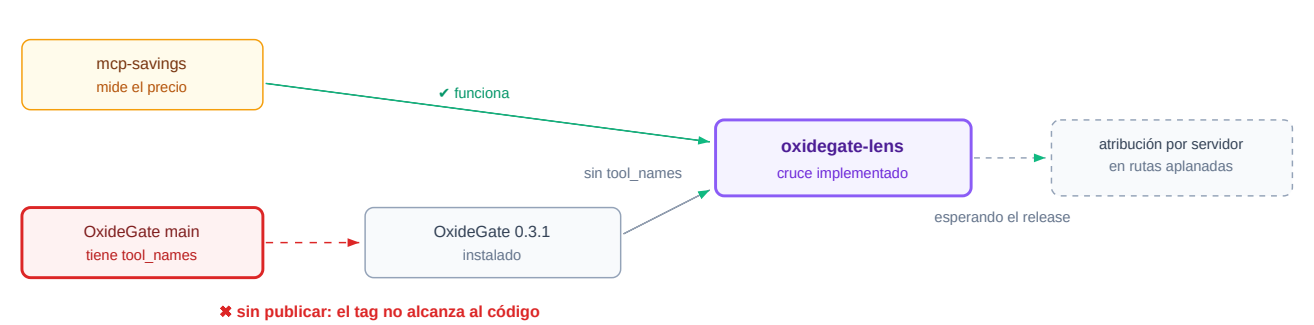
Verificado contra el snapshot real

Con el `~/.config/mcp-savings/snapshot.json` de la máquina de desarrollo — `engram` con 18 tools, `context7` con 2— y un cable aplanado:

```
engram      uses=5    in-use
context7    uses=0    already-off
```

`delegation_list` , presente en el cable, **no se atribuyó a nadie**. La tercera fila de la tabla de arriba, comprobada.

12. Lo que queda abierto



#	Asunto	Estado
OxideGate#19	Publicar un release de OxideGate que lleve <code>tool_names</code>	Lo único que bloquea de verdad. El código está en <code>main</code> (<code>6c9b98a</code>); el último tag sigue siendo <code>v0.3.1</code>
lens#13	Bajar la válvula a granularidad de tool	Anotado. Prioridad baja, y antes hay que comprobar si es accionable
—	Hook <code>tool.execute.after</code> sin verificar contra OpenCode real	Necesita sesión viva
—	El fetch-patch solo reescribe la URL exacta de Codex; los modelos no-Codex no pasan por el proxy	Producto
—	Ese patch no se distribuye: está escrito a mano en la máquina del mantenedor	Producto
—	Fase 5 en <code>mcp-savings</code> : <code>saveSnapshot</code> atómico y retirar <code>panel.ts</code>	Otro repositorio

Cerrados en esta ronda: [lens#6](#) (la matriz de harnesses), [lens#7](#) (`pi` no necesita válvula — resolvió el coste MCP por arquitectura, con un *gateway* de una sola tool y conexión bajo demanda), [lens#12](#) (configuración por proyecto con modelo de confianza) y [lens#5](#) — **el techo de la §4, levantado en código.**

La lens está lista y el proxy no. El cruce funciona, degrada correctamente cuando el campo falta, y no hará nada visible hasta que OxideGate publique una versión que lo lleve. Es el mismo patrón que dejó el monitor vacío durante meses: el código tiene el arreglo, el tag no lo alcanza, y quien lo usa sigue ejecutando lo anterior.

Seguimiento en el [project «OxideGate + Lens»](#).

Cerrado desde la primera versión de este informe: la matriz de harnesses ya no confunde *medible* con *utilizable* (párrafo con lo medido, no columna con conjeturas), y la fila de `pi.dev` pasó a «verificado».

13. La pregunta de producto que sale de todo esto

La válvula informada se diseñó para responder "*¿qué MCP puedo desconectar porque no lo uso?*". Medido: **esa pregunta no se puede responder en los dialectos que usas hoy**, porque el cable no conserva la atribución.

Lo que sí se puede responder, y con precisión:

- **Cuánto cuesta cada servidor MCP** (medido, por servidor).
- **Cuánto cuesta el conjunto de tools nativas** — que en `pi` es 3,3 veces mayor que todo el MCP junto.
- **Qué está conectado ahora mismo y qué se desconectó al arrancar.**

Esa segunda línea puede ser la más valiosa y no estaba en el diseño original.

Documento generado a partir de mediciones reales del 25 de julio de 2026. Ninguna cifra es estimada.

Cómo se regenera este documento

El PDF no es un binario huérfano: se genera del `.md` que tienes al lado.

```
node scripts/informe-a-html.mjs docs/INFORME-VALVULA-MCP.md /tmp/informe.html
google-chrome --headless --disable-gpu --no-sandbox \
  --print-to-pdf=docs/INFORME-VALVULA-MCP.pdf --no-pdf-header-footer /tmp/informe.html
```

`scripts/informe-a-html.mjs` es un conversor acotado al subconjunto de Markdown que usa este informe, sin dependencias. Los diagramas Mermaid del `.md` (que GitHub renderiza solo) se sustituyen por **SVG embebido** en el HTML, porque no hay `mermaid-cli` instalado y un Chrome headless sin red no traería un CDN. Los dos formatos muestran los mismos diagramas por caminos distintos.